

A importância do algoritmo
MERGESORT

QUAL É O SEU CONCEITO?

O Merge Sort é um algoritmo de ordenação baseado no paradigma Dividir para Conquistar, que funciona da seguinte maneira:

1. Divisão: O array é dividido recursivamente ao meio até que cada subarray contenha apenas um elemento.
2. Conquista: Os subarrays são ordenados de forma recursiva.
3. Fusão: Os subarrays ordenados são combinados de volta, mantendo a ordem crescente ou decrescente.

- Dividimos em dois:
[38, 27, 43] [3, 9, 82, 10]
- Dividimos novamente:
[38] [27, 43] [3, 9] [82, 10]
- Continuamos dividindo até que tenhamos arrays com um único elemento:
[38] [27] [43] [3] [9] [82] [10]

VANTAGENS



- **Estável** – Mantém a ordem relativa dos elementos iguais, o que pode ser útil em certas aplicações.
- **Complexidade garantida** – Tem um tempo de execução $O(n \log n)$ no pior, médio e melhor caso.
- **Bom para grandes volumes de dados** – Funciona bem para grandes conjuntos de dados, especialmente quando armazenados em disco.
- **Eficiente para listas encadeadas** – Pois não requer acesso aleatório a elementos como no primeiro exemplo.
- **Paralelizável** – Significa que ele pode ser executado de forma eficiente em múltiplos núcleos de um processador ou até mesmo em diferentes máquinas.

DESVANTAGENS



- **Alto uso de memória** – Não é in-place, precisa de $O(n)$ memória extra para armazenar as subdivisões do array, o que pode ser um problema em sistemas com pouca RAM.
- **Menos eficiente para pequenos conjuntos de dados** – Para listas pequenas, algoritmos como Insertion Sort costumam ser mais rápidos devido ao menor overhead. (custos adicionais que o algoritmo gera além do processo principal.)
- **Pode ser mais lento na prática** – Apesar da complexidade $O(n \log n)$, em alguns cenários, como com arrays pequenos ou quando os dados já estão parcialmente ordenados, QuickSort ou HeapSort podem ser mais rápidos.



COMPLEXIDADES



- Melhor caso: (array já ordenado) → O algoritmo ainda precisa dividir e mesclar as metades.
 $O(n \log n)$
- Caso médio: (ordem aleatória) → O processo de divisão e mesclagem ocorre da mesma forma.
 $O(n \log n)$
- Pior caso: (array invertido) → O algoritmo também executa todas as divisões e mesclagens.
 $O(n \log n)$

PSEUDOCÓDIGO:



Merge(vetor, esquerda, direita):

 Inicializar índices para esquerda, direita, e vetor

 Enquanto ambos os índices de esquerda e direita não estiverem fora dos limites:

 Se o elemento da esquerda for menor que o da direita:

 Coloque o elemento da esquerda no vetor

 Incrementar o índice da esquerda

 Caso contrário:

 Coloque o elemento da direita no vetor

 Incrementar o índice da direita

 Quando um dos índices atingir o limite, copie os elementos restantes do outro vetor.

MergeSort(vetor):

 Se o tamanho do vetor > 1:

 meio = tamanho do vetor / 2

 esquerda = vetor[0...meio-1]

 direita = vetor[meio...tamanho-1]

 MergeSort(esquerda)

 MergeSort(direita)

 Merge(vetor, esquerda, direita)

ESTRUTURA

```
#include <stdio.h>

// Função para mesclar duas metades ordenadas
void merge(int arr[], int left, int mid, int right) {

    int i, j, k;

    int n1 = mid - left + 1;

    int n2 = right - mid;

    int L[n1], R[n2];

    for (i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    i = 0;
    j = 0;
    k = left;
```

```
while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
        arr[k] = L[i];
        i++;
    } else {
        arr[k] = R[j];
        j++;
    }
    k++;
}
```

```
while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}
```

```
while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}
}
```



ESTRUTURA

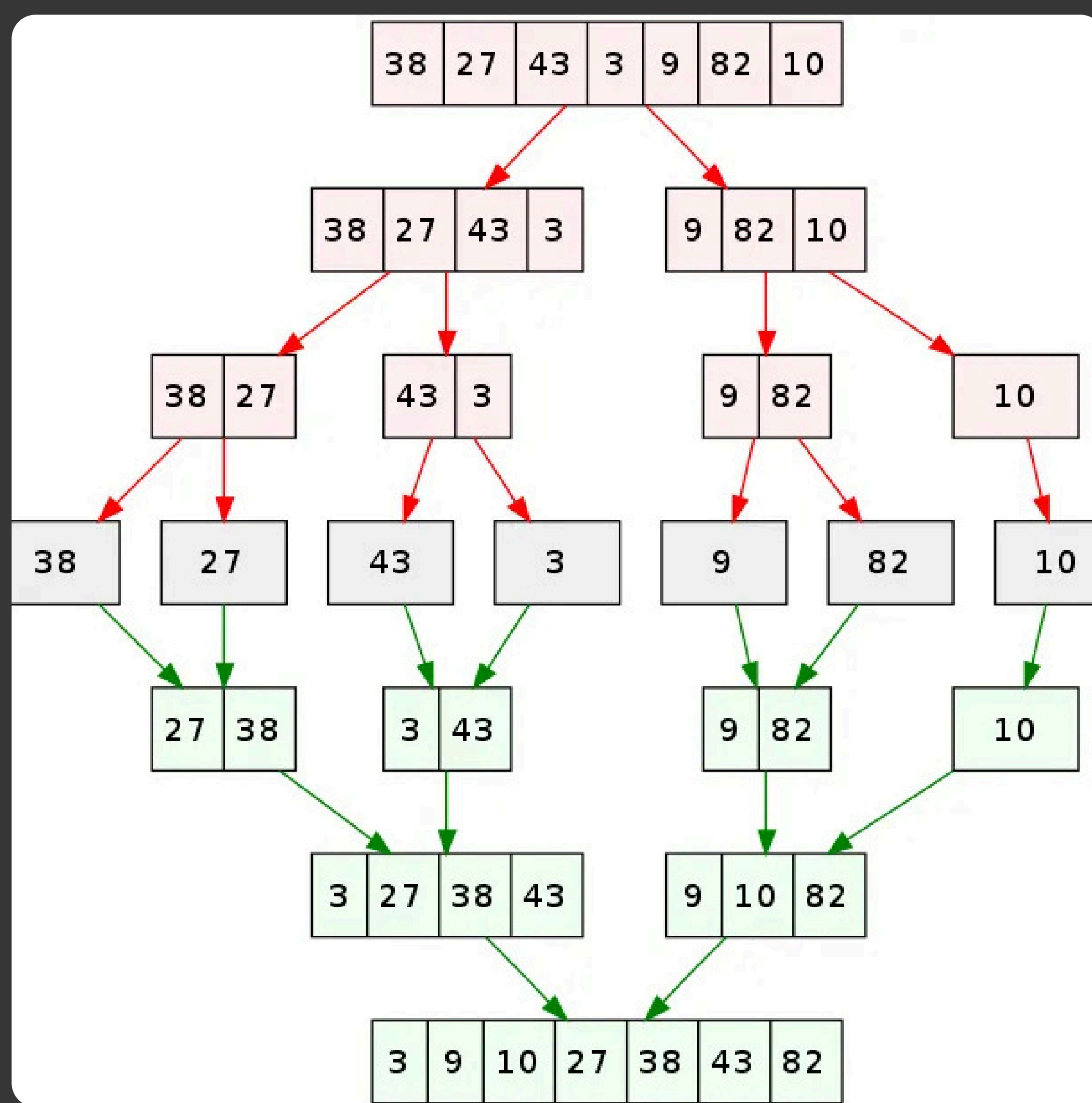
```
void mergeSort(int vetor[], int esquerda, int
direita) {
    if (esquerda < direita) {
        int meio = esquerda + (direita - esquerda) /
2;

        mergeSort(vetor, esquerda, meio); //
Ordenar a primeira metade

        mergeSort(vetor, meio + 1, direita); //
Ordenar a segunda metade

        merge(vetor, esquerda, meio, direita); //
Mesclar as duas metades
    }
}
```







Como observamos na imagem...

1. Comércio Eletrônico – Ordenação de Preços de Produtos

Um site de e-commerce pode usar o Merge Sort para organizar produtos pelo preço, do menor para o maior, garantindo que os clientes possam visualizar as opções mais baratas primeiro.

2. Educação – Ordenação de Notas dos Alunos

Uma escola pode precisar ordenar as notas dos alunos para gerar um ranking de desempenho, organizando do maior para o menor e identificando os melhores colocados.





EXERCÍCIOS

1. Como o array é dividido no Merge Sort?
2. O que acontece quando um subarray tem apenas um elemento?
3. O Merge Sort é um algoritmo iterativo ou recursivo?
4. Por que o Merge Sort precisa de memória extra?





RESOLUÇÃO

1. Como o array é dividido no Merge Sort?

→ O array é continuamente dividido ao meio até que cada parte tenha apenas um elemento.

2. O que acontece quando um subarray tem apenas um elemento?

→ Ele é considerado ordenado e começa a ser combinado com outros subarrays.

3. O Merge Sort é um algoritmo iterativo ou recursivo?

→ Ele é um algoritmo recursivo, pois chama a si mesmo para dividir o array.

4. Por que o Merge Sort precisa de memória extra?

→ Porque ele cria arrays temporários para armazenar os subarrays durante o processo de mesclagem.



Repositório



Link: <https://github.com/kayran-teixeira/Merge-Sort---UniEvangelica>



Integrantes :

- João Marcus Fleury Siqueira - 2211142;
- Deusmair Junio Souza Prado - 2310074;
- Ian Correa Couto - 2310137;
- Eduardo de Oliveira Araújo - 2310173;
- Rafael Dias Ferreira Santos - 2310165;
- André Filho Nunes Dos Santos - 2310176;
- Lucas Bastos Franco - 2310622;
- Wanderley Pereira de Souza Neto - 2313397;
- Kauã Barbosa Gonçalves - 2311131;
- Kayran Henrique Reis Teixeira - 2310370

